1 Workflow: dataset preprocessing and running experiments

1.1 Dataset preprocessing (KuaiRec)

The KuaiRec raw data must be preprocessed before training. The provided script in `dataset/kuairec/kuairec_process.py`:

- merges multiple KuaiRec feature CSVs into a single table,
- builds vocabularies for sparse/sequence features,
- constructs a feature `description` schema used by models and dataloaders,
- splits into `train`/`test`,
- writes `kuairec_data.pkl` containing the processed data and schema.

It also produces duration buckets and per-bucket play-time quantiles used by the D2Q baseline.

1.2 Code snippet: preprocessing outputs (D2Q support)

```
# dataset/kuairec/kuairec_process.py (illustrative excerpt)
n_bins = 50
df["duration_bucket"], bins = pd.qcut(df["duration"], q=n_bins, labels=False,
                                      retbins=True, duplicates="drop")
desc.append(("duration_bucket", len(bins) - 1, "spr"))

# Per-bucket play-time quantiles for D2Q mapping
quantile_num = 100
quantiles = np.linspace(0, 1, quantile_num + 1)
quantile_df = (df_train.groupby("duration_bucket")["play_time"]
              .quantile(quantiles).reset_index())
quantile_df.pivot(index="duration_bucket", columns="quantile",
                  values="play_time").to_csv(
    "d2q_duration_bucket_playtime_quantiles.csv"
)
```

1.3 Training / evaluation entrypoints

Each method has a `run_*.py` entrypoint. Typical usage is:

- `python run_egmn.py --dataset_name kuairec`
- `python run_cread.py --dataset_name kuairec`
- `python run_d2q.py --dataset_name kuairec`

These scripts:

- load `./dataset/kuairec/kuairec_data.pkl` through `dataloader/kuairec.py`,
- train for a fixed number of epochs,
- evaluate on the test set,
- print MAE, XAUC, and KL divergence.

1.4 Code snippet: EGMN training loop structure

```
# run_egmn.py (illustrative excerpt)
for epoch in range(1, args.epoch + 1):
    for features, label in dataloaders["train"]:
        pi, lambda_, mu, sigma = model(features)
        nll, reg, ent = model.loss(label.float(), pi, lambda_, mu, sigma,
                                   features["duration"].view(-1, 1))
        loss = nll + args.alpha * ent + args.beta * reg
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

1.5 Dependencies

The README indicates the project was tested with Python 3.x and PyTorch (e.g. 2.1.0), and expects common scientific Python packages (NumPy, pandas, scikit-learn).